

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:22:54

Conversation ID: 6a9e4f12-d1e0-83ee-ae4b-0798f9dab101

Conversation

Assistant

If your VM is Linux/Ubuntu and CMake says , you mainly need to install a C++ toolchain. 1. Install the compiler + build tools On Ubuntu/Debian: This installs: — C++ compiler — C compiler standard C/C++ development libraries Then verify: 2. Check where CMake sees the compiler You should get something like: Then clean your existing CMake build: If isn't available Since you've previously worked on a VM where you didn't have sudo, check: If any of those exists, tell me the output and we can configure CMake to use it without installing anything. If none exist, you'll need either: admin/root access to install , or a preinstalled compiler/toolchain somewhere on the VM. Don't install random CMake packages yet—the important missing piece is the C++ compiler (/), not CMake itself.

Code Block 1 (text)

```
sudo apt update
sudo apt install -y build-essential cmake
```

Code Block 2 (text)

```
g++ --version
gcc --version
cmake --version
make --version
```

Code Block 3 (text)

```
which g++
which gcc
```

Code Block 4 (text)

```
/usr/bin/g++
/usr/bin/gcc
```

Code Block 5 (text)

```
rm -rf build
mkdir build
cd build
cmake ..
```

Code Block 6 (text)

```
which g++  
which clang++  
which gcc  
which clang
```

User

CMake Error at CMakeLists.txt:36 (message): libpcap/npcap development files not found.

Assistant

Yep — now CMake is finding your C++ compiler, but your project also requires libpcap development files. On an Ubuntu/Debian VM, install: Then verify: You should see both files. Then clean and rerun CMake: So your VM needs these packages For your TLS fingerprinting/libpcap C++ project, I'd install: If your project has GoogleTest tests, you may also need: If is already installed Run: and: Send me that output plus your lines around 20–45, and I can tell you exactly why your CMake check is failing.

Code Block 1 (text)

```
sudo apt update  
sudo apt install -y libpcap-dev
```

Code Block 2 (text)

```
ls /usr/include/pcap.h  
ls /usr/lib/x86_64-linux-gnu/libpcap.so
```

Code Block 3 (text)

```
rm -rf build  
mkdir build  
cd build  
cmake ..
```

Code Block 4 (text)

```
sudo apt install -y \  
build-essential \  
cmake \  
libpcap-dev \  
git
```

Code Block 5 (text)

```
sudo apt install -y libgtest-dev
```

Code Block 6 (text)

```
dpkg -l | grep libpcap
```

Code Block 7 (text)

```
find /usr -name 'pcap.h' -o -name 'libpcap.so*' 2>/dev/null
```

User

CMake Error at /usr/share/cmake-4.2/Modules/FindPackageHandleStandardArgs.cmake:290 (message):
Could NOT find OpenSSL, try to set the path to OpenSSL root folder in the system variable
OPENSSL_ROOT_DIR (missing: OPENSSL_CRYPTO_LIBRARY OPENSSL_INCLUDE_DIR) Call Stack (most
recent call first): /usr/share/cmake-4.2/Modules/FindPackageHandleStandardArgs.cmake:654
(_FPHSA_FAILURE_MESSAGE) /usr/share/cmake-4.2/Modules/FindOpenSSL.cmake:752
(find_package_handle_standard_args) CMakeLists.txt:48 (find_package)

Assistant

Same issue pattern: CMake is working, but the OpenSSL development files aren't installed. On Ubuntu/
Debian, install: Then verify: You should see all of them. Then clean and rerun CMake: At this point, your
VM should have For your TLS fingerprinting project: If CMake still says OpenSSL isn't found after
installing , run: and send me the output. The issue would then likely be that OpenSSL is installed in a
nonstandard location rather than simply missing.

Code Block 1 (text)

```
sudo apt update  
sudo apt install -y libssl-dev
```

Code Block 2 (text)

```
openssl version  
ls /usr/include/openssl/ssl.h  
ls /usr/lib/x86_64-linux-gnu/libcrypto.so  
ls /usr/lib/x86_64-linux-gnu/libssl.so
```

Code Block 3 (text)

```
rm -rf build  
mkdir build  
cd build  
cmake ..
```

Code Block 4 (text)

```
sudo apt install -y \  
build-essential \  
cmake \  
libpcap-dev \  
libssl-dev \  
git
```

Code Block 5 (text)

```
dpkg -l | grep libssl  
which openssl  
find /usr -name 'libcrypto.so*' -o -name 'ssl.h' 2>/dev/null | head -30
```

User

Ign:1 http://in.archive.ubuntu.com/ubuntu resolute-updates/main arm64 libssl-dev arm64
3.5.5-1ubuntu3.3 Err:1 http://security.ubuntu.com/ubuntu resolute-updates/main arm64 libssl-dev arm64
3.5.5-1ubuntu3.3 404 Not Found [IP: 91.189.92.22 80] 404 Not Found [IP: 185.125.190.82 80] Error: Failed
to fetch http://security.ubuntu.com/ubuntu/pool/main/o/openssl/libssl-dev_3.5.5-1ubuntu3.3_arm64.deb
404 Not Found [IP: 185.125.190.82 80] Error: Unable to fetch some archives, maybe run apt update or try
with --fix-missing?

Assistant

This is not an OpenSSL problem. has stale package metadata: it is trying to download a package version that is no longer at the mirror, hence the . Run these in order: Then: If you still get a 404, check your Ubuntu release: and your configured repositories: One thing I notice Your VM is using Ubuntu (25.04) and ARM64: So if this is a VM/container image that hasn't been updated recently, the repository metadata can easily be out of sync with the mirrors. After , check what version is actually available: You want something like: Then install that candidate. Don't manually download the from somewhere else yet. First refresh the repository metadata; that's the normal fix for this particular 404.

Code Block 1 (text)

```
sudo apt clean  
sudo rm -rf /var/lib/apt/lists/*  
sudo apt update
```

Code Block 2 (text)

```
sudo apt install -y libssl-dev
```

Code Block 3 (text)

```
cat /etc/os-release
```

Code Block 4 (text)

```
grep -R "^[^#].*ubuntu" /etc/apt/sources.list /etc/apt/sources.list.d/ 2>/dev/null
```

Code Block 5 (text)

```
resolute-updates  
arm64
```

Code Block 6 (text)

```
apt-cache policy libssl-dev
```

Code Block 7 (text)

```
Candidate: 3.x.x-...
```

User

(base) ❏ cpp git:(main) sudo ./build/tlsfp_engine -i any -f "tcp port 443" Password: Sorry, try again.
Password: Sorry, try again. Password: [*] Listening on interface: any [*] Capture initialized. Datalink type:
12 [-] Warning: Redis database unavailable; fingerprints will be reported as unknown. [+] Captured
ClientHello | Flow: [192.168.64.7]:59566 -> [151.101.105.91]:443 | Reassembled Size: 2355 bytes ❏ ❏[SNI]
merino.services.mozilla.com ❏ ❏[JA3 String]
771,4865-4867-4866-49195-49199-52393-52392-49196-49200-49171-49172-156-157-47-53,0-23-65281-10-1
1-16-5-34-18-51-42-43-13-45-28-27-65037-41,4588-29-23-24-25-256-257,0 ❏ ❏[JA3 Hash]
64e49f50135a401c35548311d27c9fa0 ❏ ❏[JA3 Match] <database unavailable> ❏ ❏[JA4]
t13d1518h2_8daaf6152771_e7cacf613b58 ❏ ❏[JA4 Match] <database unavailable> [+] Captured
ServerHello | Flow: [151.101.105.91]:443 -> [10.7.15.147]:45356 | Reassembled Size: 1221 bytes ❏ ❏[JA3S
String] 771,4865,43-51-41 ❏ ❏[JA3S Hash] fcb2d4d0991292272fcb1e464eedfd43 ❏ ❏[JA3S Match]
<database unavailable> ❏ ❏[JA4S] t130300_1301_ac87d89493dd ❏ ❏[JA4S Match] <database unavailable>
[+] Captured ServerHello | Flow: [151.101.105.91]:443 -> [192.168.64.7]:59566 | Reassembled Size: 1221
bytes ❏ ❏[JA3S String] 771,4865,43-51-41 ❏ ❏[JA3S Hash] fcb2d4d0991292272fcb1e464eedfd43 ❏ ❏[JA3S
Match] <database unavailable> ❏ ❏[JA4S] t130300_1301_ac87d89493dd ❏ ❏[JA4S Match] <database
unavailable> I am getting this in Mac as well, but then it took a lot of time. Is it not capturing everything or
something

Assistant

Yes — it is capturing traffic, but your output strongly suggests that you're not seeing every packet. And

there are actually several separate things happening here. 1. is working This line is important: On Linux, is a special pseudo-interface. Datalink type is (Linux cooked capture). So your engine is successfully receiving packets from . And this proves the capture → TCP reassembly → TLS parsing pipeline is working: You successfully reconstructed a 2355-byte ClientHello and 1221-byte ServerHello. 2. But your filter does NOT mean "capture every packet" Your BPF: means: Give me packets whose TCP source OR destination port is 443. That's correct for TLS-over-TCP. However, your program then has its own filtering/reassembly logic. For example, you may capture: but your output only prints when your application recognizes a complete TLS message: So not printing a packet ≠ not capturing a packet. 3. This is the biggest thing to check Run tcpdump alongside your engine. Terminal 1: Terminal 2: Then generate traffic: Compare them. If tcpdump shows: while your engine only prints: that's completely normal if your engine is intentionally only logging TLS events. 4. But there's something suspicious in your output Look at this: and then: You're getting two ServerHello events for apparently different flows. That's worth investigating. Also, your first ClientHello is: while the ServerHello is: That's good. But you also have: That suggests you're potentially dealing with multiple network interfaces / address paths / duplicated traffic, especially on a VM. 5. Why is Mac taking a long time? This is probably the more important question. On macOS, is not equivalent to Linux's . Linux has a real special capture device: macOS generally doesn't provide the same pseudo-interface. If your program accepts: on macOS, what happens depends on how your code handles it. You should check: You'll probably see interfaces such as: For normal Wi-Fi traffic, it's usually: rather than: 6. There is another very important possibility: your engine is scanning too much From your earlier tcpdump testing, you were looking at: If your engine is processing every packet and maintaining TCP state/ reassembly for every flow, then performance can degrade considerably. Your pipeline is approximately: The expensive part is potentially: especially if you keep old flows indefinitely. 7. Test whether capture itself is slow This is the cleanest experiment. Run: Then separately: Generate traffic: If tcpdump immediately sees packets but your engine takes seconds to produce ClientHello, then the problem is inside your engine, not libpcap. If both are slow, you're simply not generating much TCP/443 traffic. 8. Also check packet/drop statistics This is very important for answering your "is it capturing everything?" question. After stopping your program, if your engine exposes libpcap stats, call: You want: If you see something like: then yes, your capture is losing packets. 9. One more thing: your Redis warning isn't affecting capture This: does not mean TLS capture failed. Your engine successfully computed: So: Your core fingerprinting pipeline is working. What I would do next On the Mac, don't use for this test. Run: and in another terminal: Then: If tcpdump shows lots of packets but your engine only reports a few ClientHello/ServerHello, that's probably expected because your engine is an event-level TLS fingerprinting engine, not a packet printer. If you want to verify whether the engine is actually dropping packets or getting stuck in reassembly, the next thing I'd inspect is your packet callback + flow table + TCP reassembly code. That will tell us whether the slowness is capture, flow lookup, reassembly, or TLS parsing.

Code Block 1 (text)

```
[*] Listening on interface: any
```

```
[*] Capture initialized. Datalink type: 12
```

Code Block 2 (text)

```
[+] Captured ClientHello  
[+] Captured ServerHello
```

Code Block 3 (text)

```
-f "tcp port 443"
```

Code Block 4 (text)

```
Client → Server  
TCP SYN  
TCP SYN-ACK  
TCP ACK  
ClientHello  
ACK  
ServerHello  
ACK  
...
```

Code Block 5 (text)

```
[+] Captured ClientHello  
[+] Captured ServerHello
```

Code Block 6 (text)

```
sudo tcpdump -i any -nn 'tcp port 443'
```

Code Block 7 (text)

```
sudo ./build/tlsfp_engine -i any -f "tcp port 443"
```

Code Block 8 (text)

```
curl https://example.com
```

Code Block 9 (text)

```
SYN  
SYN-ACK  
ACK  
PSH,ACK  
ACK  
PSH,ACK  
...
```

Code Block 10 (text)

```
ClientHello  
ServerHello
```

Code Block 11 (text)

```
[+] Captured ServerHello | Flow: [151.101.105.91]:443 -> [10.7.15.147]:45356
```

Code Block 12 (text)

```
[+] Captured ServerHello | Flow: [151.101.105.91]:443 -> [192.168.64.7]:59566
```

Code Block 13 (text)

```
192.168.64.7:59566 -> 151.101.105.91:443
```

Code Block 14 (text)

```
151.101.105.91:443 -> 192.168.64.7:59566
```

Code Block 15 (text)

```
151.101.105.91:443 -> 10.7.15.147:45356
```

Code Block 16 (text)

```
any
```

Code Block 17 (text)

```
-i any
```

Code Block 18 (text)

```
ifconfig
```

Code Block 19 (text)

```
lo0  
en0  
en1  
...
```

Code Block 20 (text)

```
sudo ./build/tlsfp_engine -i en0 -f "tcp port 443"
```

Code Block 21 (text)

```
sudo ./build/tlsfp_engine -i any
```

Code Block 22 (text)

```
sudo tcpdump -i en0 -n "port 443"
```

Code Block 23 (text)

```
NIC
↳
libpcap
↳
BPF: tcp port 443
↳
packet parser
↳
flow lookup
↳
TCP reassembly
↳
TLS parser
↳
fingerprint generation
↳
Redis lookup
↳
printing
```

Code Block 24 (text)

```
packet ↳ flow ↳ reassembly
```

Code Block 25 (text)

```
sudo tcpdump -i en0 -nn -c 100 'tcp port 443'
```

Code Block 26 (text)

```
sudo ./build/tlsfp_engine -i en0 -f "tcp port 443"
```

Code Block 27 (text)

```
curl https://example.com
curl https://www.mozilla.org
```

Code Block 28 (text)

```
struct pcap_stat stats{};
```

```
if (pcap_stats(handle, &stats) == 0) {  
std::cout << "Packets received: " << stats.ps_recv << '\n';  
std::cout << "Packets dropped: " << stats.ps_drop << '\n';  
}
```

Code Block 29 (text)

```
Packets received: 12345  
Packets dropped: 0
```

Code Block 30 (text)

```
Packets received: 50000  
Packets dropped: 15000
```

Code Block 31 (text)

```
[!] Warning: Redis database unavailable; fingerprints will be reported as unknown.
```

Code Block 32 (text)

```
JA3 Hash: 64e49f50135a401c35548311d27c9fa0  
JA4: t13d1518h2_8daaf6152771_e7cacf613b58
```

Code Block 33 (text)

```
capture      ☒  
TCP reassembly ☒  
TLS parsing  ☒  
JA3          ☒  
JA4          ☒  
Redis lookup ☒
```

Code Block 34 (text)

```
sudo tcpdump -i en0 -nn 'tcp port 443'
```

Code Block 35 (text)

```
sudo ./build/tlsfp_engine -i en0 -f "tcp port 443"
```

Code Block 36 (text)

```
curl https://example.com
```